

3. Xây Dựng Cấu Trúc Dự Án Khoa Học

Kỹ thuật phân chia component hợp lý

1. Mục tiêu

- **Hiểu rõ tầm quan trọng của việc phân chia component** trong phát triển ứng dụng React về mặt tái sử dụng, bảo trì và hiệu suất.
- **Áp dụng nguyên tắc "tái sử dụng"** để xác định khi nào nên tách một phần giao diện thành component riêng và đặt chúng vào thư mục chung (components).
- **Tránh việc phân chia component quá mức** gây ra sự phức tạp không cần thiết và giảm hiệu quả phát triển.
- **Nhận diện các dấu hiệu cho thấy một component nên được phân chia** do độ phức tạp về chức năng hoặc kỹ thuật.
- **Bước đầu hình thành tư duy phân chia component hợp lý** để xây dựng cấu trúc ứng dụng React rõ ràng và dễ bảo trì.

2. Nội dung bài giảng

2.1 Tại sao cần phân chia Component?

- **Đặt vấn đề:**
 - **Giao diện phức tạp:** Giới thiệu một giao diện phức tạp (ví dụ trang chủ website thương mại điện tử) và thảo luận về việc nếu không phân chia component thì code sẽ như thế nào (khó đọc, khó bảo trì, khó tái sử dụng)...
 - **Ghi chú cho giảng viên:** Chuẩn bị sẵn hình ảnh một trang web phức tạp và hình ảnh minh họa thư mục component lộn xộn để trình chiếu. Đặt câu hỏi gợi mở để dẫn dắt vào vấn đề về cả phân chia component và tổ chức folder.
- **Tầm quan trọng của phân chia component và tổ chức folder:**
 - **Tái sử dụng:** Nhấn mạnh lợi ích của tái sử dụng component (giảm code trùng lặp, tăng tốc độ, nhất quán UI).
 - **Bảo trì:** Giải thích lợi ích của việc code dễ đọc, dễ hiểu, dễ bảo trì khi dự án lớn.
 - **Hiệu suất:** (Sơ qua) Phân chia component đúng cách giúp re-render hiệu quả.
 - **Tổ chức và Quản lý:** Tổ chức folder tốt giúp dự án dễ điều hướng, tìm kiếm, và quản lý khi có nhiều files và components.
 - **Ghi chú cho giảng viên:** Liên hệ với ví dụ xây nhà module và việc sắp xếp đồ đạc ngăn nắp để học viên dễ hình dung lợi ích của cả phân chia component lẫn tổ chức folder.

2.2 Khi nào nên tách Component? - Nguyên tắc "Tái sử dụng" là Kim chỉ nam

- **Component tái sử dụng là gì?**
 - **Định nghĩa:** Component dùng ở nhiều nơi, nhận props khác nhau.
 - **Ví dụ minh họa:** `Button` , `Input field` , `Card sản phẩm` .
- **Ghi chú cho giảng viên:** Chuẩn bị sẵn code ví dụ về `Button` và `ProductCard Component`. Trình chiếu code, giải thích cách props giúp tái sử dụng component. **Nhấn mạnh:** "Nhận diện component lặp đi lặp lại trên nhiều trang web, đó là component tái sử dụng."
- **Thực hành nhận diện:** Yêu cầu học viên chỉ ra component tái sử dụng trong giao diện mẫu.
 - **Ghi chú cho giảng viên:** **Chuẩn bị giao diện website đơn giản.** Chia nhóm, yêu cầu nhận diện component tái sử dụng, thảo luận và đưa ra đáp án.

2.3 Thư mục "components"

- **Thư mục `components`:** Quy ước đặt các component tái sử dụng vào `src/components` .

- Giới thiệu Cấu trúc Folder `src/components` :
 - **Mô tả:** Tất cả component đặt trực tiếp trong thư mục `components` .

```
src/
├── components/
│   ├── Button.jsx
│   ├── Button.css
│   ├── Card.jsx
│   ├── Card.css
│   ├── Input.jsx
│   ├── Input.css
│   └── ...
```

- **Ghi chú cho giảng viên:** Trình chiếu cấu trúc folder. Kèm theo code 1 vài ví dụ.

2.4 Tránh phân chia Component quá mức

- **Vấn đề của việc phân chia quá nhỏ:**
 - Tăng phức tạp, khó quản lý, khó theo dõi luồng dữ liệu.
 - Giảm hiệu quả phát triển (tốn thời gian, lợi ích không đáng kể).
- **Khi nào KHÔNG nên tách component:**
 - Chỉ dùng một nơi.
 - Quá nhỏ, tách không làm code dễ đọc hơn.
 - Tách làm cấu trúc dự án phức tạp hơn.

2.5 Phân chia Component do yếu tố kỹ thuật và độ phức tạp

- **Component quá lớn, phức tạp:** Tách để dễ quản lý, kiểm thử.
- **Tối ưu hiệu suất:** (Sẽ nói sau) Tách để tối ưu re-render (nhắc sơ lược `React.memo` , `useMemo` , `useCallback`).

Render Có Điều Kiện trong React

1. Mục tiêu

- **Hiểu rõ khái niệm và tầm quan trọng của render có điều kiện** trong việc xây dựng giao diện người dùng động và tương tác trong React.
- **Nắm vững các phương pháp render có điều kiện phổ biến trong JSX**, bao gồm `if/else` , toán tử ba ngôi (`? :`), toán tử logic AND (`&&`) và toán tử logic OR (`||`).
- **Áp dụng thành thạo các phương pháp render có điều kiện** để hiển thị các phần tử UI khác nhau dựa trên các điều kiện khác nhau (ví dụ: trạng thái đăng nhập, dữ liệu tải, v.v.).

2. Nội dung bài giảng

2.1 Tại sao cần Render có điều kiện?

- **Đặt vấn đề:**
 - **Giao diện tĩnh vs. Giao diện động:** So sánh giao diện web tĩnh (HTML thuần túy) với giao diện web động (ví dụ: Facebook, Gmail). Nhấn mạnh sự cần thiết của việc hiển thị giao diện khác nhau dựa trên hành động người dùng, dữ liệu, trạng thái ứng dụng, v.v.
 - **Ví dụ minh họa:**
 - Ẩn/hiện nút "Đăng nhập" và "Đăng xuất" dựa trên trạng thái đăng nhập của người dùng.
 - Hiển thị loading spinner trong khi chờ dữ liệu tải về.
 - **Tầm quan trọng của Render có điều kiện:** Giúp tạo ra trải nghiệm người dùng linh hoạt, tương tác và phản ánh đúng trạng thái ứng dụng.

- **Ghi chú cho giảng viên:** Sử dụng các ví dụ trực quan, gần gũi với thực tế để học viên dễ dàng hình dung được tầm quan trọng của render có điều kiện. Có thể đặt câu hỏi gợi mở: "Nếu chúng ta muốn hiển thị nút 'Đăng xuất' chỉ khi người dùng đã đăng nhập, chúng ta làm như thế nào?"

2.2 Các phương pháp Render có điều kiện trong JSX

- **Sử dụng `if/else` statements:**
 - **Giải thích:** `if/else` là cú pháp JavaScript cơ bản, có thể được sử dụng bên ngoài phần `return` của component để quyết định biến JSX nào sẽ được render.

```
function MyComponent({ isLoggedIn }) {
  let content;
  if (isLoggedIn) {
    content = <div>Chào mừng bạn đã đăng nhập!</div>;
  } else {
    content = <p>Vui lòng đăng nhập để xem nội dung.</p>;
  }

  return (
    <div>
      {content}
      {/* ... phần JSX khác ... */}
    </div>
  );
}
```

- **Ưu điểm:** Rõ ràng, dễ đọc với logic điều kiện phức tạp.
 - **Nhược điểm:** Hơi dài dòng, tách biệt logic điều kiện khỏi phần JSX render.
- **Sử dụng Toán tử ba ngôi (Ternary Operator - `? :`)**
 - **Giải thích:** Toán tử ba ngôi là cú pháp ngắn gọn để viết `if/else` trong một dòng, thường được sử dụng trực tiếp trong JSX.

```
function MyComponent({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <div>Chào mừng bạn đã đăng nhập!</div>
      ) : (
        <p>Vui lòng đăng nhập để xem nội dung.</p>
      )}
      {/* ... phần JSX khác ... */}
    </div>
  );
}
```

- **Ưu điểm:** Ngắn gọn, tiện lợi cho các điều kiện đơn giản.
 - **Nhược điểm:** Khó đọc nếu logic điều kiện trở nên phức tạp hoặc lồng ghép nhiều toán tử ba ngôi.
- **Sử dụng Toán tử logic AND (`&&`)**
 - **Giải thích:** `&&` operator trong JavaScript trả về vế thứ hai nếu vế thứ nhất là `truthy`, ngược lại trả về vế thứ nhất (`falsy`). Trong JSX, có thể dùng `&&` để render một phần tử chỉ khi điều kiện là đúng.

```
function MyComponent({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn && <div>Chào mừng bạn đã đăng nhập!</div>}
      {/* Phần tử div chỉ render khi isLoggedIn là true */}
      {/* ... phần JSX khác ... */}
    </div>
  );
}
```

```
    </div>
  );
}
```

- **Ưu điểm:** Cực kỳ ngắn gọn để render một phần tử khi điều kiện đúng.
- **Nhược điểm:** Chỉ dùng được cho trường hợp "render khi đúng", không có nhánh "else". Cần cẩn thận khi điều kiện là số `0`, vì `0` là falsy và sẽ không render gì cả (có thể hiển thị số `0` nếu render trực tiếp số).

- **Sử dụng Toán tử logic OR (`||`)**

- **Giải thích:** `||` operator trong JavaScript trả về về thứ nhất nếu về thứ nhất là `truthy`, ngược lại trả về về thứ hai. Trong JSX, có thể dùng `||` để cung cấp giá trị fallback hoặc render một phần tử khác nếu điều kiện là sai.

```
function MyComponent({ userName }) {
  return (
    <div>
      <h1>Xin chào, {userName || 'khách!'}</h1>
      {/* Hiển thị userName nếu có, nếu không có thì hiển thị 'khách!' */}
      {/* ... phần JSX khác ... */}
    </div>
  );
}
```

- **Ưu điểm:** Ngắn gọn để cung cấp giá trị mặc định hoặc fallback.
 - **Nhược điểm:** Ít phổ biến hơn cho render có điều kiện phức tạp, thường dùng để cung cấp giá trị mặc định.
- **Ghi chú cho giảng viên:** Trình bày từng phương pháp render có điều kiện, giải thích cú pháp, ưu nhược điểm và ví dụ code minh họa. Sử dụng ví dụ đơn giản, dễ hiểu. Khuyến khích học viên thực hành code trực tiếp với từng phương pháp.

Cách Đặt State Để Tối Ưu Re-render và Hiệu Suất

1. Mục tiêu

- **Hiểu rõ cơ chế re-render của React** và khi nào component re-render.
- **Nhận thức được rằng component con sẽ re-render khi component cha re-render** (theo mặc định).
- **Hiểu được tác động của việc đặt state ở component cha** đối với hiệu suất re-render của toàn bộ cây component con.
- **Nắm vững nguyên tắc "colocate state"** (đặt state gần nơi sử dụng nhất) và lý do tại sao nó giúp tối ưu re-render.
- **Áp dụng các kỹ thuật đặt state hợp lý** để giảm thiểu re-render không cần thiết và cải thiện hiệu suất ứng dụng React.
- **Bước đầu hình thành tư duy về tối ưu hiệu suất React** thông qua việc quản lý state thông minh.

2. Nội dung bài giảng:

2.1 Re-render trong React - Điều gì xảy ra khi Component "vẽ lại"?

- **Đặt vấn đề:**
 - **Giải thích "Re-render" là gì:** "Re-render" (vẽ lại) là quá trình React cập nhật DOM (Document Object Model) khi có sự thay đổi trong component. Khi component re-render, function component sẽ được gọi lại (hoặc phương thức `render()` trong class component), JSX sẽ được tạo lại, và React sẽ so sánh Virtual DOM cũ và mới để cập nhật những thay đổi cần thiết lên DOM thật.
 - **Khi nào Component Re-render?**
 - **State thay đổi:** Đây là nguyên nhân phổ biến nhất. Khi `setState` được gọi (hoặc setter function từ `useState` hook), component và các component con của nó sẽ được lên lịch re-render.

- **Quan trọng:** Khi một component cha re-render, tất cả các component con của nó cũng sẽ re-render theo mặc định.
- **Tầm quan trọng của việc hiểu cơ chế Re-render:** Hiểu rõ khi nào component re-render giúp chúng ta viết code React hiệu quả hơn, tránh re-render không cần thiết, và tối ưu hiệu suất ứng dụng.
- **Ghi chú cho giảng viên:** Sử dụng hình ảnh minh họa quá trình re-render của React, từ state/props change → component re-render → Virtual DOM diffing → DOM update..

2.2 Component Cha Re-render -> Component Con Re-render - Minh họa bằng Cây Component

- **Vẽ cấu trúc cây Component đơn giản:**
 - **Ví dụ cây component:** Vẽ lên bảng hoặc slide một cấu trúc cây component đơn giản, ví dụ: `App` → `ParentComponent` → `ChildComponent1`, `ChildComponent2` .
 - **Giải thích:**
 - `App` là component gốc (root).
 - `ParentComponent` là component cha của `ChildComponent1` và `ChildComponent2` .
 - `ChildComponent1` và `ChildComponent2` là component con của `ParentComponent` .
 - **Ghi chú cho giảng viên:** Thực hiện thao tác vẽ cây component trực tiếp trên bảng hoặc slide để học viên dễ hình dung. Sử dụng hình vẽ đơn giản, dễ hiểu, không cần quá phức tạp. Có thể dùng các hình hộp chữ nhật hoặc hình tròn để biểu diễn component, và mũi tên để biểu diễn quan hệ cha-con.
- **Minh họa Re-render khi State ở Component Cha thay đổi:**
 - **Đặt state ở `ParentComponent`** : Giả sử chúng ta đặt một state (ví dụ `count`) trong `ParentComponent` .
 - **Giải thích:** Khi state `count` trong `ParentComponent` thay đổi (ví dụ khi click button trong `ParentComponent` để tăng `count`), điều gì sẽ xảy ra?
 - `ParentComponent` sẽ re-render (chắc chắn rồi, vì state của nó thay đổi).
 - Quan trọng: `ChildComponent1` và `ChildComponent2` cũng sẽ re-render, mặc dù chúng không trực tiếp sử dụng state `count` và props của chúng có thể không thay đổi.
 - **Lý do:** Theo cơ chế mặc định của React, khi component cha re-render, React sẽ re-render tất cả các component con của nó để đảm bảo giao diện nhất quán và phản ánh đúng state mới nhất của ứng dụng.
 - **Ghi chú cho giảng viên:** Vẽ mũi tên lan tỏa re-render từ `ParentComponent` xuống `ChildComponent1` và `ChildComponent2` trên sơ đồ cây component.
- **Ví dụ Code minh họa:**

```
// ParentComponent.jsx
import React, { useState } from 'react';
import ChildComponent1 from './ChildComponent1';
import ChildComponent2 from './ChildComponent2';

function ParentComponent() {
  const [count, setCount] = useState(0);

  console.log('ParentComponent re-rendered!'); // Log để theo dõi re-render

  const incrementCount = () => {
    setCount(count + 1);
  };

  return (
    <div style={{ border: '1px solid blue', padding: '20px', margin: '10px' }}>
      <h2>Parent Component</h2>
      <p>Count: {count}</p>
      <button onClick={incrementCount}>Increment Count</button>
      <ChildComponent1 />
      <ChildComponent2 />
    </div>
  );
}
```

```
    </div>
  );
}
```

```
export default ParentComponent;
```

```
// ChildComponent1.jsx & ChildComponent2.jsx (tương tự nhau)
import React from 'react';

function ChildComponent1() {
  console.log('ChildComponent1 re-rendered!'); // Log để theo dõi re-render
  return (
    <div style={{ border: '1px solid green', padding: '10px', margin: '10px' }}>
      <h3>Child Component 1</h3></div>
    );
}

export default ChildComponent1;
```

```
// App.jsx (ví dụ sử dụng)
import React from 'react';
import ParentComponent from './ParentComponent';

function App() {
  return (
    <div>
      <h1>App Component</h1>
      <ParentComponent />
    </div>
  );
}

export default App;
```

- **Ghi chú cho giảng viên:** Chuẩn bị sẵn ví dụ code này. Trình chiếu code và chạy ứng dụng. Mở console của trình duyệt và thực hiện thao tác "Increment Count" trong `ParentComponent`. Quan sát log console để thấy rõ:
 - Mỗi khi click "Increment Count", cả `ParentComponent`, `ChildComponent1`, và `ChildComponent2` đều log "re-rendered!".
 - Điều này chứng minh rằng **component con re-render khi component cha re-render**, ngay cả khi state thay đổi chỉ ở component cha.*

2.3 Đặt State ở đâu cho hợp lý? - Nguyên tắc "Colocate State"

- **Vấn đề của việc KHÔNG đặt State tại Component Cha Chung Gần Nhất:**
 - **Re-render lan rộng và không cần thiết:** Nếu state được đặt ở một component cha **quá cao** trong cây component (ví dụ component gốc `App`) trong khi **chỉ có một nhóm nhỏ component con ở nhánh cây con nào đó thực sự cần state này**, thì khi state thay đổi, **toàn bộ nhánh cây component con** (bao gồm cả những component con *không liên quan* đến state đó) sẽ bị re-render theo cơ chế mặc định của React. Điều này gây ra re-render lan rộng không cần thiết và lãng phí tài nguyên.
 - **Giảm hiệu suất do re-render lãng phí:** Việc re-render một lượng lớn component không cần thiết sẽ tiêu tốn tài nguyên tính toán của trình duyệt, làm chậm ứng dụng, đặc biệt là trong các ứng dụng lớn, phức tạp, hoặc khi re-render diễn ra thường xuyên.
 - **Khó bảo trì và theo dõi luồng dữ liệu:** Khi state được đặt quá xa nơi sử dụng, việc theo dõi luồng dữ liệu trở nên phức tạp hơn. "Prop drilling" (truyền props qua nhiều tầng component trung gian không cần thiết) có thể xảy ra, làm code rối rắm và khó bảo trì.
- **Nguyên tắc "Colocate State" (Đặt State gần nơi sử dụng nhất):**

- **Giải thích:** Nguyên tắc Colocate State (Đặt State Cùng Vị Trí) có nghĩa là đặt state ở vị trí thấp nhất có thể trong cây component, nhưng vẫn phải đảm bảo rằng component chứa state đó là component cha chung gần nhất của tất cả các component thực sự cần truy cập và sử dụng state đó.
- **Ví dụ:**
 - Nếu state chỉ dùng để điều khiển hiển thị/logic của `ChildComponent1`, hãy đặt state trực tiếp vào `ChildComponent1`, **không cần đặt ở `ParentComponent`**.
 - Nếu state cần dùng chung cho cả `ParentComponent` và `ChildComponent1`, thì `ParentComponent` là vị trí hợp lý để đặt state.
- **Lợi ích của "Colocate State":**
 - **Giảm re-render không cần thiết:** Chỉ những component thực sự cần thiết mới re-render khi state thay đổi.
 - **Tăng hiệu suất:** Ứng dụng chạy nhanh và mượt mà hơn.
 - **Code dễ quản lý hơn:** Logic và state của component được "gói gọn" trong component đó, dễ hiểu và dễ bảo trì.
- **Ghi chú cho giảng viên:**
 - **Vẽ cấu trúc cây component** để minh họa rõ ràng quan hệ cha-con.
 - **Nhấn mạnh việc xác định các component thực sự sử dụng state.**
 - **Hướng dẫn học viên tìm và khoanh vùng component cha chung gần nhất** của nhóm component đó trên sơ đồ cây.
 - **Giải thích rằng vị trí tối ưu cho state là ở component cha chung gần nhất này.**
 - Minh họa hậu quả khi đặt state **cao hơn** component cha chung gần nhất, dẫn đến re-render không cần thiết cho các nhánh cây con khác.
 - Sử dụng ví dụ code và sơ đồ để học viên trực quan hóa khái niệm và dễ dàng áp dụng nguyên tắc "Colocate State".

2.4 Khi nào thì "Colocate State" KHÔNG phù hợp? - Khi State cần dùng chung

- **Tình huống State cần dùng chung:**
 - **Ví dụ:** State `theme` (chủ đề giao diện) cần được sử dụng bởi nhiều component khác nhau trong ứng dụng (header, sidebar, content, footer...).
 - **Giải thích:** Trong trường hợp này, việc "colocate state" vào một component con cụ thể sẽ không phù hợp. Vì state `theme` cần được truy cập và có thể thay đổi từ nhiều nơi.
- **Giải pháp: Nâng State lên Component Cha chung (Lifting State Up):**
 - **Giải thích:** Khi state cần dùng chung, hãy **nâng state lên component cha chung gần nhất của tất cả các component cần sử dụng state đó.**
 - **Ví dụ:** Nếu `theme` cần dùng cho `Header` và `Sidebar`, và cả hai đều là component con của `App`, thì hãy đặt state `theme` ở component `App`.
 - **Mô hình dữ liệu "Luồng dữ liệu một chiều (One-way data flow)":** Dữ liệu (state) thường đi từ component cha xuống component con thông qua props. Component con không nên trực tiếp sửa đổi state của component cha (trừ khi thông qua callback function được truyền từ cha xuống con).
- **Công cụ quản lý state toàn cục (Giới thiệu sơ bộ - sẽ đi sâu hơn ở bài nâng cao):**
 - **Context API, Redux, Zustand, Recoil...:** Khi ứng dụng lớn và state cần chia sẻ rộng rãi, việc "lifting state up" có thể trở nên phức tạp. Các công cụ quản lý state toàn cục giúp quản lý state dễ dàng hơn trong các ứng dụng lớn. (Giới thiệu tên các công cụ, không đi sâu vào chi tiết ở bài cơ bản).
- **Ghi chú cho giảng viên:** Giải thích rõ ràng tình huống khi "colocate state" không còn phù hợp (state dùng chung). Giới thiệu khái niệm "lifting state up" và "one-way data flow". Nhắc đến ngắn gọn các công cụ quản lý state toàn cục để học viên có cái nhìn tổng quan, nhưng nhấn mạnh rằng bài học này tập trung vào "colocate state" và "lifting state up" cơ bản.

useEffect trong React: Quản lý Vòng Đời Component Cơ Bản

1. Mục tiêu

- **Hiểu rõ khái niệm "side effects"** (tác dụng phụ) trong React và tại sao cần quản lý chúng.
- **Nắm vững mục đích và vai trò của Hook `useEffect`** trong việc xử lý side effects trong functional component.
- **Hiểu cách `useEffect` thay thế các lifecycle methods** (ví dụ: `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`) của class component trong functional component.
- **Nắm vững cú pháp cơ bản của `useEffect`**, bao gồm callback function và dependency array.
- **Sử dụng `useEffect` để thực hiện các tác vụ side effects phổ biến**, như:
 - Gọi API để lấy dữ liệu (data fetching).
 - Thiết lập subscriptions (đăng ký) và timers (hẹn giờ).
 - Thao tác trực tiếp với DOM API (trong trường hợp cần thiết).
 - Logging.
- **Hiểu rõ vai trò và tác dụng của dependency array** trong việc kiểm soát thời điểm `useEffect` hook được thực thi.
 - Sử dụng dependency array rỗng (`[]`) để mô phỏng `componentDidMount` và `componentWillUnmount`.
 - Sử dụng dependency array với các giá trị state/props để mô phỏng `componentDidUpdate`.
 - Hiểu trường hợp không sử dụng dependency array (và những lưu ý khi sử dụng).
- **Nắm vững khái niệm và cách sử dụng cleanup function** trong `useEffect` để tránh memory leaks và các side effects không mong muốn.

2. Nội dung bài giảng:

2.1 Side Effects và Vòng Đời Component trong React

- **Side Effects là gì?**
 - **Giải thích "Side Effects"**: Trong lập trình, "side effects" là những tác động "ngoài luồng" của một function. Trong React component, side effects là những hành động **tác động ra bên ngoài component** hoặc **gây ra thay đổi không liên quan trực tiếp đến việc render giao diện**, ví dụ:
 - **Data fetching (Gọi API)**: Lấy dữ liệu từ server.
 - **DOM manipulation**: Thay đổi DOM trực tiếp (ít khi dùng trong React, nhưng có trường hợp cần thiết).
 - **Timers (`setTimeout`, `setInterval`)**: Thiết lập hẹn giờ hoặc lặp lại hành động theo thời gian.
 - **Logging**: Ghi log ra console hoặc server.
 - **Subscriptions**: Đăng ký theo dõi sự kiện bên ngoài (ví dụ WebSockets, event listeners của trình duyệt).
 - **Tại sao cần quản lý Side Effects?** Side effects là cần thiết để component tương tác với thế giới bên ngoài, nhưng nếu không quản lý đúng cách, chúng có thể gây ra các vấn đề như:
 - **Memory leaks**: Rò rỉ bộ nhớ nếu không dọn dẹp các subscriptions hoặc timers khi component unmount.
 - **Unwanted behavior**: Hành vi không mong muốn do side effects được thực thi không đúng thời điểm hoặc lặp lại không kiểm soát.
 - **Khó debug**: Khó gỡ lỗi nếu side effects gây ra lỗi ngầm hoặc không rõ ràng.
- **Vòng Đời Component (Component Lifecycle)**:
 - **Vòng đời của Class Component (ôn lại)**: Nhắc lại các lifecycle methods chính trong class component: `constructor`, `render`, `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`. Giải thích vai trò của từng method trong các giai đoạn khác nhau của vòng đời component (mounting, updating, unmounting).
 - **Functional Component và Hook `useEffect`**: Giới thiệu rằng functional component không có lifecycle methods như class component. `useEffect` Hook được sinh ra để cung cấp khả năng thực hiện side effects và quản lý vòng đời component trong functional component. `useEffect` kết hợp chức năng của `componentDidMount`, `componentDidUpdate`, và `componentWillUnmount` trong một API duy nhất.
- **Ghi chú cho giảng viên**: Bắt đầu bằng việc giải thích khái niệm "side effects" một cách dễ hiểu, sử dụng các ví dụ quen thuộc. Sau đó ôn lại vòng đời component trong class component để tạo liên kết và làm nổi bật vai trò

của `useEffect` trong functional component. Sử dụng hình ảnh sơ đồ vòng đời class component để minh họa.

2.2 Cú pháp cơ bản của `useEffect` Hook

- Cú pháp cơ bản:

```
useEffect(() => {
  // Callback function (chứa logic side effect)

  return () => {
    // Cleanup function (tùy chọn)
  };
}, [/* Dependency array (tùy chọn) */]);
```

- Giải thích các thành phần:

- `useEffect(callback, dependencyArray)` : `useEffect` là một Hook, được gọi trực tiếp trong functional component. Nó nhận hai đối số:
 - `callback function (bắt buộc)`**: Đây là function chứa logic side effect mà bạn muốn thực thi (ví dụ: gọi API, thiết lập timer...). `callback` function này sẽ được thực thi *sau mỗi lần render* của component (theo mặc định, sẽ được kiểm soát bởi dependency array).
 - `dependencyArray (tùy chọn)`**: Đây là một array các giá trị mà `useEffect` sẽ "theo dõi" sự thay đổi. Dependency array quyết định **khi nào** `useEffect` callback function sẽ được thực thi:
 - Nếu `dependencyArray` không được truyền vào (undefined)**: `useEffect` callback function sẽ chạy sau mỗi lần render của component.
 - Nếu `dependencyArray` là một array rỗng (`[]`)**: `useEffect` callback function sẽ chạy chỉ một lần sau lần render đầu tiên (tương tự `componentDidMount`) và chạy cleanup function khi component unmount (tương tự `componentWillUnmount`).
 - Nếu `dependencyArray` chứa các giá trị (state, props)**: `useEffect` callback function sẽ chạy sau lần render đầu tiên và chạy lại sau mỗi lần render nếu có bất kỳ giá trị nào trong dependency array thay đổi so với lần render trước đó.
 - `cleanup function (tùy chọn, return từ callback function)`**: Nếu `callback` function trả về một function khác, function này được gọi là cleanup function. Cleanup function sẽ được thực thi:
 - Trước khi `useEffect` callback function được thực thi lại (trong các lần re-render tiếp theo)**.
 - Khi component unmount (bị hủy khỏi DOM).
 - Mục đích của cleanup function là để "dọn dẹp" các side effects đã thiết lập ở lần thực thi trước đó**, ví dụ: hủy timers, hủy subscriptions, remove event listeners... để tránh memory leaks và các vấn đề khác.

- Ví dụ code đơn giản:

```
import React, { useState, useEffect } from 'react';

function ExampleComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('useEffect is running!'); // Side effect: logging
    document.title = `Count: ${count}`; // Side effect: DOM manipulation
  }); // Không có dependency array - chạy sau mỗi render

  const increment = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>Count: {count}</p>
    </div>
  );
}
```

```
    <button onClick={increment}>Increment</button>
  </div>
);
}

export default ExampleComponent;
```

- **Ghi chú cho giảng viên:** Giải thích cú pháp `useEffect` một cách cẩn thận, tập trung vào callback function và dependency array. Sử dụng ví dụ code đơn giản để minh họa cách `useEffect` chạy sau mỗi render khi không có dependency array. Chạy ví dụ code và mở console trình duyệt để học viên thấy log "useEffect is running!" xuất hiện sau mỗi lần click button (re-render).

2.3 Sử dụng useEffect cho các Side Effects Phổ Biến

- **Mô phỏng componentDidMount (chạy một lần sau render đầu tiên):**
 - **Dependency array rỗng ([]):** Sử dụng dependency array rỗng để `useEffect` callback chỉ chạy một lần sau lần render đầu tiên, tương tự `componentDidMount` trong class component.
 - **Ví dụ: Data Fetching khi component mount:**

```
import React, { useState, useEffect } from 'react';

function DataFetchingComponent() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true); // Bắt đầu loading
    fetch('https://api.example.com/data') // Thay bằng API thật
      .then(response => {
        if (!response.ok) {
          throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json();
      })
      .then(data => {
        setData(data); // Cập nhật data
        setError(null); // Xóa lỗi (nếu có)
        setLoading(false); // Kết thúc loading
      })
      .catch(error => {
        setError(error); // Cập nhật lỗi
        setData(null); // Xóa data (nếu có)
        setLoading(false); // Kết thúc loading
        console.error('Error fetching data:', error);
      });
  }, []); // Dependency array rỗng - chỉ chạy 1 lần khi mount

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  if (!data) return <p>No data to display.</p>;

  return (
    <div>
      <h1>Data from API</h1>
      <pre>{JSON.stringify(data, null, 2)}</pre>
    </div>
  );
}
```

```
export default DataFetchingComponent;
```

- **Ghi chú cho giảng viên:** Giải thích cách dependency array rằng mô phỏng `componentDidMount`. Sử dụng ví dụ data fetching là một use case điển hình. Thay URL API ví dụ bằng một API public thật (ví dụ JSONPlaceholder) để có thể chạy demo trực tiếp. Chạy ví dụ code và cho học viên thấy data được fetch và hiển thị sau lần render đầu tiên.
- **Mô phỏng `componentDidUpdate` (chạy khi dependencies thay đổi):**
 - **Dependency array với các giá trị state/props:** Sử dụng dependency array chứa các giá trị state hoặc props để `useEffect` callback chạy lại mỗi khi **có bất kỳ giá trị nào trong dependency array thay đổi** so với lần render trước, tương tự `componentDidUpdate` trong class component.
 - **Ví dụ: Gọi API lại khi prop `userId` thay đổi:**

```
import React, { useState, useEffect } from 'react';

function UserProfileComponent({ userId }) { // Nhận prop userId
  const [userData, setUserData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    setUserData(null); // Reset data khi userId thay đổi
    fetch(`https://api.example.com/users/${userId}`) // Fetch user data dựa trên userId
      .then(response => {
        if (!response.ok) {
          throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json();
      })
      .then(data => {
        setUserData(data);
        setError(null);
        setLoading(false);
      })
      .catch(error => {
        setError(error);
        setUserData(null);
        setLoading(false);
        console.error('Error fetching user data:', error);
      });
  }, [userId]); // Dependency array chứa userId - chạy lại khi userId thay đổi

  if (loading) return <p>Loading user data...</p>;
  if (error) return <p>Error: {error.message}</p>;
  if (!userData) return <p>No user data found.</p>;

  return (
    <div>
      <h2>User Profile</h2>
      <pre>{JSON.stringify(userData, null, 2)}</pre>
    </div>
  );
}

export default UserProfileComponent;
```

- **Ghi chú cho giảng viên:** Giải thích cách dependency array với giá trị mô phỏng `componentDidUpdate` . Sử dụng ví dụ fetch user profile data và API call lại khi `userId` prop thay đổi. Tạo một component cha (`UserProfileContainer`) để quản lý state `currentUserId` và truyền xuống `UserProfileComponent` như prop `userId` . Cho học viên thấy khi thay đổi `currentUserId` ở component cha, `UserProfileComponent` re-render và gọi API lại để fetch user data mới.

- **Mô phỏng `componentWillUnmount` (cleanup khi component unmount):**

- **Cleanup function:** Sử dụng cleanup function (function được return từ `useEffect` callback) để thực hiện các tác vụ dọn dẹp khi component unmount, tương tự `componentWillUnmount` trong class component.
- **Ví dụ: Thiết lập và hủy timer:**

```
import React, { useState, useEffect } from 'react';

function TimerComponent() {

  const [time, setTime] = useState(0);

  useEffect(() => {
    console.log('useEffect for timer is setting up!');
    const timerId = setInterval(() => {
      setTime(prevTime => prevTime + 1); // Cập nhật time mỗi giây
    }, 1000);

    return () => { // Cleanup function
      console.log('Cleanup function is running!');
      clearInterval(timerId); // Hủy timer khi component unmount
    };
  }, []); // Dependency array rỗng - thiết lập timer chỉ 1 lần khi mount

  return (
    <div>
      <h2>Timer Component</h2>
      <p>Time elapsed: {time} seconds</p>
    </div>
  );
}

export default TimerComponent;
```

- **Ghi chú cho giảng viên:** Giải thích vai trò của cleanup function trong việc dọn dẹp side effects, đặc biệt là timers và subscriptions, để tránh memory leaks. Sử dụng ví dụ timer để minh họa cách thiết lập timer trong `useEffect` callback và hủy timer trong cleanup function. Sử dụng component cha (`TimerContainer`) để mount và unmount `TimerComponent` sau một khoảng thời gian nhất định (ví dụ dùng `setTimeout` ở component cha để unmount component con) để học viên thấy cleanup function chạy khi component con unmount.

2.4 Dependency Array - Đi sâu vào chi tiết

- **Tổng kết vai trò của Dependency Array:** Dependency array là trái tim của `useEffect` , nó quyết định **khi nào và bao nhiêu lần** `useEffect` callback function sẽ được thực thi.
- **Các cấu hình Dependency Array và tác dụng:**
 - **[] (Empty array):** Chạy **một lần khi mount, cleanup khi unmount** - Mô phỏng `componentDidMount` và `componentWillUnmount` . Thường dùng cho:
 - Data fetching ban đầu.
 - Thiết lập subscriptions/timers chạy suốt vòng đời component và cần dọn dẹp khi unmount.
 - **undefined hoặc không truyền (No array):** Chạy **sau mỗi render** - Cần thận khi sử dụng vì có thể gây infinite loop nếu side effect bên trong callback làm thay đổi state và gây re-render liên tục. **Thường ít khi dùng trực tiếp**, nhưng có thể hữu ích trong một số trường hợp đặc biệt (ví dụ logging, đo performance...).

- `[value1, value2, ...]` **(Array chứa giá trị)**: Chạy **khi mount lần đầu và khi bất kỳ giá trị nào trong array thay đổi** (shallow comparison). Mô phỏng `componentDidUpdate` (có điều kiện). Thường dùng cho:
 - Gọi API hoặc thực hiện side effects dựa trên sự thay đổi của props hoặc state cụ thể.
 - Phản ứng với sự thay đổi của các giá trị bên ngoài component.
-

Tổng Hợp Kiến Thức

Bài Tập Cuối Chương

Đáp Án Bài Tập Cuối Chương